

PRD를 에이전트에게 전달하기

CLAUDE.md는 프로젝트의 헌법 — OODA 루프와 Permissions

— 본 자료의 구성

01 AI 에이전트와 대화하는 법

02 CLAUDE.md 4영역

03 OODA 루프와 Permissions

— SECTION 1

AI 에이전트와 대화하는 법

- 01 컨텍스트의 정체를 설명한다
- 02 무상태성이 만드는 문제를 인식한다
- 03 컨텍스트 앵커링의 효과를 이해한다

DEFINITION

컨텍스트

에이전트가 명령을 처리할 때
참조하는 정보의 집합.

- 에이전트가 코드를 잘 쓰게 만드는 일은 명령의 기술이 아니라 **컨텍스트**의 기술이다.
- 같은 명령이라도 **컨텍스트**가 다르면 결과가 다르다.
- **컨텍스트**가 비어 있으면 임의 추측, 명확하면 일관된 해석.

첫 단정

CLAUDE.md는 프로젝트의 헌법이다

의미

매 세션 자동 로드 — 모든 명령이 그 위에서 작동

출발점

단정의 근거는 에이전트의 무상태성에 있다

DEFINITION

에이전트는 매 세션마다 백지 상태로 시작한다

어제 작업한 내용을 오늘 기억하지 않고,
인간 동료처럼 맥락을 이어가지 않는다.

- 어제 작업한 내용을 오늘 기억 못 함
- 다른 폴더의 작업을 이 폴더로 가져오지 못함
- 새 동료를 매일 처음 만나는 상황과 같다 — 설명이 매번 다르면 동료의 작업도 매번 다르다

— CLAUDE.MD 운영의 효과

한 번의 작성이 모든 명령의 컨텍스트가 된다

무상태성을 보완하는 가장 단순한 방법 — 자동 로드되는 헌법.

운영 방식	비용 / 효과
CLAUDE.md 없음	매 명령마다 컨텍스트 수동 입력. 토큰 누적
CLAUDE.md 있음	한 번 작성. 모든 명령이 그 위에서 작동

컨텍스트 앵커링

정보를 한 곳에 고정하고, 모든 명령이 그 고정점을 참조하 게 만드는 구조

효과

고정점이 명확할수록 에이전트의 작업이 일관된다

위험

고정점이 흐릿하면 작업도 흐릿하다

안티패턴

"쇼핑몰 만들어줘"

너무 광범위한 명령. 에이전트가 추측 모드로 진입.

이유 페르소나·핵심 기능·기술 스택 모두 빈 채로 출발

회피 컨텍스트 없이 명령만 받음
매번 다른 결과 (LLM 출력 분산)
결과: 재작업 반복

컨텍스트 앵커링

"CLAUDE.md를 본 상태에서 회원가입 화면을 만들어줘"

고정점이 명확한 명령 — 헌법 위에서 작동.

이유 페르소나·기능·스택이 자동 로드 컨텍스트로 작동

결과 명령은 짧아져도 결과는 일관 — 컨텍스트가 큰 짐을 진다

핵심 용어 정리

컨텍스트

에이전트가 명령을 처리할 때 참조하는 정보의 집합

컨텍스트 앵커링

핵심 정보를 자동 로드되게 하는 기법

무상태성

매 세션마다 백지 상태로 시작하는 성질

자동 로드

매 세션 시작 시 자동으로 읽힘

— SECTION 2

CLAUDE.md 4영역

- 01 CLAUDE.md 4영역을 나열한다
- 02 각 영역의 역할을 설명한다
- 03 AI 검증 루프로 명확성을 확인한다

4영역 한눈에 보기

CLAUDE.md = 4영역 × 200줄 미만

매 명령마다 자동 로드 — 길수록 비용 증가. 200줄 미만이 균형.

#	영역	답하는 질문	분량
1	프로젝트 개요	무엇을 만드는가	한 단락
2	기술 스택과 파일 구조	어떤 도구로 만드는가	5-7줄
3	코딩 가이드라인	어떻게 작성하는가	5-10줄
4	제약사항	무엇을 안 하는가	3-5줄

DEFINITION

PRD의 요약 — 한 단락

페르소나, 핵심 기능 3개, 성공 지표.
자세한 PRD는 docs/prd.md 참조 한 줄.

- PRD 전체를 복사하지 않는다 — 매 명령 자동 로드 비용 증가.
- **요약** + 참조 한 줄이 균형.

— 시드 6번 사례 · 1영역 (프로젝트 개요)

1영역 — 한 단락 + 참조 한 줄

월 매출 100-500만원 1인 스마트스토어 셀러 대상.
상품 사진과 키워드 5개를 입력하면
차별화 카피 3개 후보를 5초 이내에 생성.
스마트스토어 형식 텍스트 출력 지원.
자세한 내용은 docs/prd.md 참조.

시드 6번 사례 · 2영역 (기술 스택과 파일 구조)

2영역 — 스택 + 폴더 구조

[기술 스택]

- FE: Next.js + Tailwind CSS
- BE: Node.js + Express
- AI API: OpenAI GPT-4o
- 상태 관리: Zustand

[폴더 구조]

- src/app/ 페이지 라우팅
- src/components/ UI 컴포넌트
- src/lib/ API 호출, 유틸
- docs/ PRD, 페르소나
- rules/ 코딩 규칙
- server/ BE 코드

— [시드 6번 사례](#) · [3영역 \(코딩 가이드라인\)](#)

3영역 — 네이밍 · 커밋 · 테스트 · 포맷

- 함수명: camelCase
- 컴포넌트명: PascalCase
- 파일명: kebab-case
- 커밋 메시지: [type] description
- 단위 테스트: Jest + React Testing Library
- 코드 포맷: Prettier + ESLint

— 시드 6번 사례 · 4영역 (제약사항)

4영역 — 의도적으로 하지 않는 것

제약사항이 명확하면 에이전트가 임의 기능을 추가하지 않는다.

- 모바일 앱 미지원 (웹만)
- 이미지 자동 생성 미지원 (텍스트만)
- 다국어 미지원 (한국어만)
- 광고 자동 집행 미지원
- 결제 기능 미포함

두 번째 단정

PRD는 무엇을 만들 것인가, CLAUDE.md는 어떻게 만들 것인가이다

분리

두 문서가 답하는 질문이 다르다

경계

PRD는 합의 문서, CLAUDE.md는 작업 절차

다섯 축에서 두 문서는 다르다

비교	PRD	CLAUDE.MD
답하는 질문	무엇을	어떻게
분량	수 페이지	200줄 미만
갱신 주기	검증 사이클 단위	빌드 진행 단위
자동 로드	안 됨	매 세션 자동
의도	합의 문서	작업 절차

흔히 놓치는 3가지 함정

CLAUDE.md 작성에서 자주 발생하는 실수

#	함정	결과
1	PRD를 통째로 복사	매 명령마다 비효율 (자동 로드 비용)
2	너무 짧게 작성	빈 영역을 에이전트가 임의로 채움
3	한 번 작성 후 갱신 안 함	빌드 진행과 어긋남

AI 검증 루프

AI가 CLAUDE.md만 읽고 프로젝트를 한 문장으로 요약할 수 있다면 헌법으로서 충분히 작동한다

본질

CLAUDE.md를 만든 뒤 즉시 검증하는 방법

판단

요약이 의도와 일치 → 통과 / 불일치 → 모호한 부분 존재

만들고 즉시 검증, 의도 일치까지 반복

1	Claude Code 명령	"이 CLAUDE.md만 보고 우리 프로젝트가 무엇인지 한 문장으로 요약해줘"
2	AI 요약 답변 확인	답변 텍스트를 그대로 읽기
3	작업자 의도와 비교	일치 → 검증 통과 / 불일치 → 모호한 부분 존재
4	불일치 시 CLAUDE.md 수정	모호한 영역 식별 → 명확한 표현으로 보강
5	재검증 (의도 일치까지 반복)	작업자 의도와 일치할 때까지 1~4 반복

4축에서 비교한 결과 — AI 검증 권장

항목	사람 검증 (기존)	AI 검증 루프 (현재)
비용	동료 시간 확보	즉시 가능
속도	한 번에 시간 소요	반복 빠름
객관성	동료의 사전 지식 영향	컨텍스트만으로 판단
본 학습 운영	비현실적	권장

시드 6번 사례 · AI 검증 통과

프롬프트 → AI 답변 → 점검

프롬프트

"이 CLAUDE.md만 보고 우리 프로젝트가 무엇인지
한 문장으로 요약해줘"

AI 답변

"1인 스마트스토어 셀러를 위한
차별화 카피 자동 생성 도구.
상품 사진과 키워드를 입력하면
5초 이내에 카피 3개를 출력."

점검 결과

OK 페르소나 일치

OK 핵심 기능 일치

OK 성공 지표 반영

OK 제약사항 반영

→ 검증 통과

안티패턴

"CLAUDE.md를 동료에게 보여줘서 30초 안에 이해되는지 물어본다"

사람 검증에 의존. 즉시 가능 X, 객관성 흔들림.

이유 동료 시간 확보 비용 + 사전 지식 영향

회피 비용·속도 비현실적
동료의 도메인 이해도가 결과 좌우
반복 비용 큼

올바른 검증

"AI에게 CLAUDE.md만 읽혀 요약 시킨다. 작업자 의도와 일치할 때까지 수정 반복"

즉시·반복·객관적. AI 검증 루프 5단계.

이유 컨텍스트만으로 판단 — 사전 지식 영향 X

결과 의도 일치까지 반복하면 모호한 부분이 자연스럽게 드러난다

핵심 용어 정리

CLAUDE.md

Claude Code가 모든 명령 전에 자동 로드하는 헌법

헌법 위반

에이전트가 CLAUDE.md 규칙을 어긴 경우

AI 검증 루프

AI에게 요약을 시켜 의도와 일치 확인

4영역

프로젝트 개요, 기술 스택, 코딩 가이드, 제약사항

200줄 미만

권장 상한. 토큰 비용 통제

— SECTION 3

OODA 루프와 Permissions

- 01 OODA 루프 4단계를 설명한다
- 02 Permissions 3단계를 구분한다
- 03 OODA와 Permissions의 연결을 인식한다

DEFINITION

의사결정의 사이클을 4단계로 정의하는 프레임워크

Observe → Orient → Decide → Act.

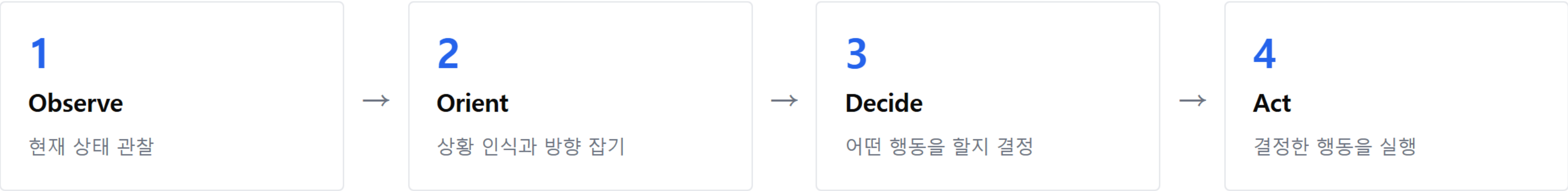
미군 전투 조종사 John Boyd가 개발.

- 본 학습에서는 Claude Code 운영 사이클로 적용.
- 에이전트의 자율과 작업자 개입의 경계를 분석하는 틀.

— OODA 4단계

Act 이후 새 상태가 다시 Observe로 순환

한 사이클은 의사결정의 한 단위 — 끊임없이 도는 순환 구조.



↳ (Act → Observe로 다시 순환)

매 명령마다 4단계를 거친다

1

Observe

· CLAUDE.md 자동 로드

· 폴더 구조 인식

· 명령 텍스트 수신

2

Orient

· 4영역 정보로 상황 정렬

· 페르소나·기술 스택 인식

· 현재 명령이 어느 기능인지 판단

3

Decide

· 어떤 도구를 쓸지 결정

· 어떤 파일을 수정할지 결정

· Permissions에 따라 분류

4

Act

· 결정한 도구 실행

· 파일 수정, 명령 실행

↳ (Act 이후 새 상태가 다시 Observe로 순환)

세 번째 단정

에이전트의 OODA 루프 중 Decide와 Act 사이에 작업자가 개입하는 지점이 Permissions이다

의미

Permissions = 자율 권한의 경계선

운영

도구 위험도에 따라 자율 / 확인 / 금지 분배

도구별 자율 권한을 세 단계로 분배

단계	영문	의미
자동 허용	auto_allow	확인 없이 사용
매번 확인	ask_before	사용 전 확인
절대 금지	deny	권한 자체 차단

— 자동 허용 (AUTO_ALLOW)

위험도 낮고 빈도 높은 도구

매번 확인하면 작업자 부담만 커진다 — 자율에 맡긴다.

도구	이유
Read	파일 읽기. 영향 없음
Grep, Glob	검색. 영향 없음
Edit	단순 코드 수정. 빈도 높음

— 매번 확인 (ASK_BEFORE)

영향 범위가 크거나 외부 비용이 발생하는 도구

매번 확인이 안전 — 작업자와 에이전트가 함께 결정.

도구	이유
Bash	셸 명령. 영향 범위 큼
Write	새 파일 작성. 신중 필요
WebFetch	외부 API. 비용 발생

절대 금지 (DENY)

잘못 사용되면 회복 어려운 결과

권한 자체를 차단 — 작업자도 안 한다는 시스템 차원의 결정.

도구	이유
BashWithSudo	시스템 명령. 회복 불가
환경 변수 노출	보안 위험

Decide → Act 사이가 작업자 개입 지점

PERMISSIONS	OODA 흐름	작업자 개입
auto_allow	Decide → 바로 Act	없음
ask_before	Decide → 확인 → Act	매번
deny	Decide → 차단	시스템 차원

— 책임 분배의 의미

Permissions = 책임 분배의 명시화

단계	책임	의미
auto_allow	에이전트	자율에 맡김
ask_before	함께	작업자 + 에이전트 결정
deny	작업자도 안 함	시스템 차단

— 두 극단의 위험

한쪽으로 치우치면 둘 다 무너진다

권장 템플릿이 안전과 속도의 균형 — 다음 슬라이드에서 확인.

극단	결과
모두 ask_before	빌드 속도 절반
모두 auto_allow	시스템 파일 위험

— 권장 SETTINGS.JSON

본 학습은 이 템플릿을 그대로 사용

자유 변형은 자가 학습 영역. 그대로 적용하면 안전과 속도의 균형이 자동.

파일 경로 **.claude/settings.json**

```
{
  "permissions": {
    "allow": ["Read", "Grep", "Glob", "Edit"],
    "ask": ["Bash", "Write", "WebFetch"],
    "deny": ["BashWithSudo"]
  }
}
```

안티패턴

"Permissions를 너무 느슨하게 두면"

모든 도구를 auto_allow에 두는 운영.

이유 에이전트가 시스템 파일까지 만지게 됨

회피 BashWithSudo도 자동 허용
환경 변수 노출 위험
시스템 단위 회복 어려운 결과

권장 운영

"권장 settings.json 템플릿을 그대로 적용"

안전과 속도의 균형이 자동으로 갖춰진다.

이유 auto_allow / ask_before / deny가 도구 위험도에 맞게 분배됨

결과 자유 변형은 자가 학습 영역 — 본 학습은 템플릿 그대로

핵심 용어 정리

OODA 루프

Observe-Orient-Decide-Act 의사결정 사이클

Permissions

자율 권한 정의. 자동 / 확인 / 금지

ask_before

사용 전 확인 필요

settings.json

프로젝트별 Claude Code 설정 파일

auto_allow

확인 없이 사용 가능

deny

권한 자체 차단